

Optimization Methods

Report on Optimal Solitaire Game Solutions using A* Search and Deadlock Analysis

S058 – Ishita Kaur Sahni

S071 – Jiya Thacker

MBA Tech (Data Science)

1. Introduction

For decades, games have served as a fertile ground for advancements in artificial intelligence and operations research. Skill-based solitaire card games like FreeCell or King Albert, where all cards are visible, are of particular interest. Unlike games of chance, these are deterministic classical planning problems. The primary challenge in finding a solution is not just winning, but finding the *optimal* solution—the one with the fewest possible moves.

This problem is computationally difficult (NP-hard), primarily due to the presence of "deadlocks." A deadlock is a game state where a card required for the goal (e.g., a 3 of Spades) is trapped in a tableau column *above* a card that must be moved after it (e.g., an 8 of Spades). Resolving this requires "extra" moves that don't directly advance the game but are necessary to unlock the stuck cards.

This paper presents a novel and efficient method for determining these provably optimal solutions. The core of the authors' contribution is the development of a highly accurate, admissible heuristic function for the A* (A-star) search algorithm. This heuristic is not just a simple card count; it is derived by modelling the game state as a directed graph. In this graph, deadlocks mathematically manifest as cycles. The heuristic's strength lies in its ability to analyse these cycles and calculate the *minimum number of moves* required to break them, providing the A* search with a precise estimate of the "cost-to-go" and guiding it to the optimal solution.

2. Detailed Methodology

The system's architecture is a synthesis of the classic A* search algorithm and a bespoke, domain-specific heuristic. The A* algorithm performs the state-space search, while the heuristic intelligently guides it.

2.1 The A* Search Algorithm

The A* (A-star) algorithm is a widely used, best-first search algorithm for finding the least-cost path from a start state to a goal state. It builds a tree of partial paths, prioritising which state to explore next based on a cost function $f(n)$. This function is a sum of two other functions:

- $g(n)$: The *known* cost of the path from the initial state to the current state n . In this problem, this is simply the number of moves already made.
- $h(n)$: The *estimated* cost of the cheapest path from state n to the goal state. This is the heuristic function.

For A* to guarantee an optimal solution, the heuristic $h(n)$ must be **admissible**, meaning it *never overestimates* the true cost to reach the goal. The paper's entire methodological contribution is the creation of a new, highly accurate, and admissible $h(n)$.

2.2 Game State to Directed Graph Mapping

The key insight is to translate the visual layout of a FreeCell game into a formal mathematical structure: a directed graph. Each of the 52 cards is treated as a node in this graph. Two distinct types of edges are created to represent the game's rules and state:

- Dependency Edges (Permanent):** These edges represent the game's fundamental goal. A directed edge is created from each card to the card of the *next lower rank of the same suit* (e.g., **8S** → **7S**, **7S** → **6S**, etc.). These edges are a permanent, static feature of the game that never changes, as the rules for "building up" the foundation remain constant.
- Blocking Edges (Temporary):** These edges represent the game's current obstacles. A directed edge is created from each card in a tableau column to the card *directly below it*. These edges are dynamic; every time a card is moved, the blocking edges are recalculated to reflect the new board state.

A **deadlock** is visually represented as a **cycle** in this graph. For example, a 1-suit cycle occurs if a 3 of Spades (3S) is in a column above an 8 of Spades (8S). This creates a cycle:

- The 3S has a path of **blocking edges** leading down to the 8S.
- The 8S has a path of **dependency edges** (**8S** → **7S** → ... → **4S** → **3S**) that loops back to the 3S. The 3S is thus "blocked" by the 8S, but the 8S cannot be played to the foundation until the 3S is. They are mutually dependent, and the cycle must be broken by moving one of them to a free cell or another column.

2.3 Cycle Determination and Filtering (The Optimisation)

A raw graph mapping of a single FreeCell game can generate an unmanageable number of cycles. The authors apply a rigorous filtering process to reduce this to only the *essential* cycles that must be analysed.

- i. **q-Suit Cycles:** Cycles are classified by how many suits' dependency edges are involved (1-suit, 2-suit, 3-suit, or 4-suit). A 1-suit cycle is the simple case described above. A 2-suit cycle might involve a 5 of Spades (5S) blocking an 8 of Clubs (8C), which in turn has dependency edges leading to a 3 of Clubs (3C) that blocks the 5S.
- ii. **Redundant Cycle Elimination:** A cycle c_1 is considered redundant and is ignored if its set of blocking edges is a *superset* of the blocking edges of another cycle c_2 . This is because any move that breaks the simpler cycle c_2 will *also* break the more complex cycle c_1 . Therefore, only the most fundamental, "non-redundant" cycles are kept.
- iii. **Relevant Edge Identification:** Only blocking edges can be removed (by moving a card). Furthermore, in any given cycle, only the *lowest* blocking edge in a column segment is "relevant." Any blocking edges *above* it cannot be moved anyway until the lower cards are cleared. This dramatically reduces the number of "candidate" edges for breaking a cycle.
- iv. **Duplicate Cycle Elimination:** After filtering, any cycles that share the same set of *relevant blocking edges* are treated as duplicates and counted only once.

This aggressive optimisation is the core of the method. For FreeCell game #1, this process reduced the 26,133 total cycles to just **87 essential, non-redundant cycles** (12 1-suit, 39 2-suit, 34 3-suit, and 2 4-suit) that actually need to be analysed.

3. Mathematical Work

The methodology is formalised into two key equations.

i. The A* Cost Function

The primary search algorithm is driven by the A* cost function:

$$f(n) = g(n) + h(n)$$

Where:

- $f(n)$ is the total estimated cost (solution length) of the path from the start to the goal, passing through state n .
- $g(n)$ is the *known* cost (number of moves) from the start state to state n .
- $h(n)$ is the *estimated* cost (heuristic) from state n to the goal.

ii. The Deadlock Heuristic Function

The paper defines its new heuristic $h(n)$ as a two-part sum:

$$h(n) = (52 - m_f(n)) + m_e(n)$$

Where:

- $(52 - m_f(n))$ is the simple baseline heuristic: the 52 total cards minus the number of cards already in the foundation ($m_f(n)$). This is the minimum number of moves left if there were no deadlocks.
- $m_e(n)$ is the crucial component: the minimum number of edges (moves) that must be made to eliminate all identified non-redundant cycles. The calculation of $m_e(n)$ is itself an optimisation problem.

The authors use a brute-force, depth-first exhaustive search to find the *smallest set of relevant blocking edges* whose removal would break *all* essential cycles. The worst-case complexity C for this search is the product of the number of relevant edges e_i in each of the m_c cycles:

$$C = \prod_{i=1}^{m_c} e_i$$

To manage this, they further optimise this search by prioritising the removal of edges that are contained in the *largest number of other cycles*, thereby breaking many cycles at once and pruning the search tree effectively. The resulting count—the size of this minimum set—is the value of $m_e(n)$.

4. Result Analysis

This section details both the high-level performance results from the paper and a practical code example demonstrating the core deadlock-finding logic.

4.1 Paper's Solver Results

4.2 The authors implemented their solver in C++ (with optimisations like Zobrist hashing for duplicate state detection) and ran it on the first 5,000 Microsoft FreeCell games on an Intel i3 processor.

i. **Heuristic Trade-off:** A key finding was the trade-off between heuristic accuracy and computation time. They created different versions of their heuristic ($h_q(n)$) based on the maximum cycle type considered (e.g., h_1 only considers 1-suit cycles, h_2 considers 1- and 2-suit cycles, etc.).

- While h_3 and h_4 were more accurate (reducing the *number of states* searched), the *time per state* to calculate these complex cycles was so high that the total solution time *increased*.
- They found the optimal balance was $h_2(n)$, which only used 1- and 2-suit cycles. All 5,000 games were solved using this heuristic.

ii. **Performance (h_2 heuristic):**

- **Success:** Optimal solutions were found for all 5,000 games tested.
- **Admissibility:** The initial estimated length was *never* greater than the actual optimal solution length, confirming the heuristic is admissible and the solutions are guaranteed to be optimal.
- **Solution Length:** Shortest: 64 moves. Average: 77 moves. Longest: 93 moves.
- **Processing Time:** Shortest: 0.4 sec. Average: 39.9 sec. Longest: 6579 sec.
- **Dominant Task:** The CPU time was overwhelmingly dominated by finding cycles and calculating $m_e(n)$, not by the A* search itself.

Table 1: Results for FreeCell Games 1-10 (using h_2 heuristic)

Game #	1-suit cycles	2-suit cycles	Length Estimate	Actual Length	Time (sec)	States Searched
1	12	39	73	82	30.8	567699
2	13	5	68	73	1.9	101186
3	15	8	70	70	0.6	20499
4	37	34	72	79	31.0	1220026
5	16	46	78	85	122.0	3687136
6	12	24	73	75	1.1	31912

7	13	39	72	76	1.7	74369
8	8	58	70	74	13.2	367784
9	19	25	77	81	2.0	77990
10	16	18	73	80	7.7	315643

This table shows the results of running the paper's optimal solver on the first 10 Microsoft FreeCell games. It's designed to show how complex the starting boards were and how well the solver performed.

Here is a breakdown of what each column means:

i. Initial state Columns

This group of columns describes the game board *before* the solver makes its first move.

- **FreeCell Game #:** This is the ID number for the specific FreeCell game, corresponding to the Microsoft game seeds (e.g., Game #1, Game #2).
- **1-suit cycles:** This shows how many "simple" deadlocks (where a card is blocked by a higher-rank card of the same suit) were found in the starting layout. For example, Game 4 started with 37 of these.
- **2-suit cycles:** This shows how many "complex" deadlocks (involving two different suits) were found. Game 8, for instance, started with 58 of these.
- **Length estimate:** This is the heuristic's "smart guess" for the shortest possible solution length, calculated *before* the search began. This number comes from adding the cards not in the foundation (52) to the minimum moves needed to break all the identified 1-suit and 2-suit cycles. For Game 1, the initial estimate was 73 moves.

ii. Solution Columns

This group of columns describes the *results* after the solver finished its work.

- **Length:** This is the actual, provably optimal number of moves in the final solution found by the A* search. For Game 1, the shortest possible solution was 82 moves.
- **Time (sec):** This is how many seconds it took the computer to find the optimal solution. Game 5 was the slowest in this list, taking 122.0 seconds.
- **States searched:** This is the total number of different game board positions (states) the A* algorithm had to explore to find the optimal solution. Game 5 also required searching the most states (3,687,136).

Key Takeaways from the Table:

- i. The Heuristic is Admissible:** The most important thing to notice is that the length estimate is never greater than the actual length. For Game 1, the estimate (73) is less

than the solution (82). This proves the heuristic is "admissible" (it never overestimates the cost), which is what *guarantees* the A* search will find the truly optimal solution.

- ii. **Accuracy Matters:** Look at Game 3. The length estimate (70) was *exactly* the same as the final length (70). Because the heuristic was perfect, the solver had to do very little work: it searched the fewest states (20,499) and finished in the fastest time (0.6 seconds). This shows the heuristic is very strong.
- iii. **Complexity Varies:** The number of initial cycles gives a rough idea of difficulty, but it's not the whole story. Game 5 took over 122 seconds and searched 3.6 million states, while Game 4, which had many 1-suit cycles, was solved much faster (31.0 seconds). This shows that the *type* and *interaction* of the deadlocks are what make the problem hard.

5. Practical Code Example: 1-Suit Cycle Detection

To demonstrate the fundamental logic, the following code implements the "identify 1-suit cycles" algorithm described in Figure 4 of the paper. It does *not* perform the A* search but focuses on analyzing a hard-coded board for the simplest deadlock type.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Solitaire Deadlock Demo</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <style>
    body { font-family: 'Inter', sans-serif; }
    .card {
      width: 80px;
      height: 120px;
      border: 1px solid #999;
      border-radius: 8px;
      background-color: white;
      display: flex;
      flex-direction: column;
      justify-content: space-between;
      padding: 8px;
      font-size: 1.25rem;
      font-weight: 600;
      cursor: grab;
      user-select: none;
      box-shadow: 0 2px 4px rgba(0,0,0,0.1);
      position: relative; /* For stacking */
    }
    .card.is-red { color: #D91823; }
    .card.is-black { color: #222; }
    .card-top { text-align: left; }
    .card-bottom { text-align: right; transform: rotate(180deg); }
    .card.highlight-problem {
      box-shadow: 0 0 15px 5px rgba(255, 0, 0, 0.7);
      border: 2px solid red;
      z-index: 20;
    }
    .card.highlight-blocker {
      box-shadow: 0 0 15px 5px rgba(255, 165, 0, 0.7);
      border: 2px solid orange;
      z-index: 10;
    }
  </style>
  .column .card {
```

```

        /* Adjusted for taller 7-card columns */
        margin-top: -95px;
    }
    .column .card:first-child {
        margin-top: 0;
    }
    .cell, .foundation {
        width: 80px;
        height: 120px;
        border: 2px dashed #AAA;
        border-radius: 8px;
        background-color: rgba(255, 255, 255, 0.1);
        position: relative;
    }
    .foundation {
        border-style: solid;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 4rem;
        color: rgba(255, 255, 255, 0.1);
    }
    .cell:not(:empty) .card, .foundation:not(:empty) .card {
        margin-top: 0;
        position: absolute;
        top: 0;
        left: 0;
    }
    .hint {
        display: none;
        background: #fff8e1;
        color: #6d4c0c;
        border: 1px solid #ffeccc;
        padding: 16px;
        border-radius: 8px;
        margin-top: 20px;
        max-width: 600px;
    }
    .card.dragging {
        opacity: 0.5;
        cursor: grabbing;
    }
    .drop-target {
        background-color: rgba(0, 255, 0, 0.1);
        border-color: #0f0;
    }
}
</style>
</head>

```

```

<body class="bg-gray-900 text-gray-100 p-6 min-h-screen flex flex-col items-center">

  <h1 class="text-3xl font-bold mb-4">Solitaire Deadlock Solver</h1>
  <!-- Updated description -->
  <p class="mb-4 text-gray-400 max-w-2xl text-center">This demo is loaded with the 52-card game from your image. You can drag and drop cards or click the button to find 1-suit deadlocks.</p>

  <div class="flex gap-4 mb-6">
    <button id="findDeadlocks" class="bg-red-600 hover:bg-red-700 text-white font-bold py-2 px-4 rounded-lg shadow-md transition-all">1. Find Deadlocks</button>
    <button id="showHint" class="bg-blue-600 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded-lg shadow-md transition-all">2. Show Hint to Solve</button>
    <button id="resetBoard" class="bg-gray-600 hover:bg-gray-700 text-white font-bold py-2 px-4 rounded-lg shadow-md transition-all">Reset Board</button>
  </div>

  <!-- Free Cells and Foundations -->
  <div class="grid grid-cols-8 gap-4 w-full max-w-4xl mb-6">
    <!-- Free Cells -->
    <div id="cell-0" class="cell dropzone" data-type="cell" data-index="0"></div>
    <div id="cell-1" class="cell dropzone" data-type="cell" data-index="1"></div>
    <div id="cell-2" class="cell dropzone" data-type="cell" data-index="2"></div>
    <div id="cell-3" class="cell dropzone" data-type="cell" data-index="3"></div>
    <!-- Foundations -->
    <div id="foundation-0" class="foundation dropzone" data-type="foundation" data-suit="0"><span class="suit-icon">♥</span></div>
    <div id="foundation-1" class="foundation dropzone" data-type="foundation" data-suit="1"><span class="suit-icon">♦</span></div>
    <div id="foundation-2" class="foundation dropzone" data-type="foundation" data-suit="2"><span class="suit-icon">♣</span></div>
    <div id="foundation-3" class="foundation dropzone" data-type="foundation" data-suit="3"><span class="suit-icon">♠</span></div>
  </div>

  <!-- Tableau Columns -->
  <div class="grid grid-cols-8 gap-4 w-full max-w-4xl">
    <div id="column-0" class="column dropzone" data-type="column" data-index="0"></div>
  </div>

```

```

        <div id="column-1" class="column dropzone" data-type="column" data-
index="1"></div>
        <div id="column-2" class="column dropzone" data-type="column" data-
index="2"></div>
        <div id="column-3" class="column dropzone" data-type="column" data-
index="3"></div>
        <div id="column-4" class="column dropzone" data-type="column" data-
index="4"></div>
        <div id="column-5" class="column dropzone" data-type="column" data-
index="5"></div>
        <div id="column-6" class="column dropzone" data-type="column" data-
index="6"></div>
        <div id="column-7" class="column dropzone" data-type="column" data-
index="7"></div>
    </div>

    <!-- Hint Box -->
    <div id="hintBox" class="hint">
        <!-- Updated Hint for the new board -->
        <h3 class="font-bold text-lg mb-2">How to Solve the AS Deadlock (The
A* Solution)</h3>
        <p>This shows a solution for a key deadlock in Column 5: the
<b>AS</b> (Ace of Spades) is stuck under a <b>3S</b>.</p>
        <ol class="list-decimal list-inside mt-2 space-y-1">
            <li>Find a place to move the <b>7H</b> (from Col 5).</li>
            <li>Find a place to move the <b>7D</b> (from Col 5).</li>
            <li>Find a place to move the <b>10H</b> (from Col 5).</li>
            <li>Find a place to move the <b>8C</b> (from Col 5).</li>
            <li>Move the <b>AC</b> (from Col 5) to the ♠ foundation.</li>
            <li>Move the <b>3S</b> (from Col 5) to a free cell or another
column.</li>
            <li>Move 7: Now the <b>AS</b> is free! Move the <b>AS</b>
(from Col 5) to the ♠ foundation.</li>
        </ol>
        <p class="mt-2 text-sm">The A* solver and the $m_e(n)$ heuristic are
what find this specific 7-move (or longer) optimal path out of all possible
moves!</p>
    </div>

    <script>
        // --- Data ---
        // Suit: 0=H, 1=D, 2=C, 3=S
        // Rank: 1=A, 2-10, 11=J, 12=Q, 13=K

        // *** THIS IS THE UPDATED INPUT FROM YOUR IMAGE ***
        const initialBoardState = {
            columns: [

```

```

        [{suit: 3, rank: 5}, {suit: 3, rank: 2}, {suit: 3, rank: 12},
{suit: 3, rank: 9}, {suit: 3, rank: 8}, {suit: 0, rank: 8}, {suit: 3, rank:
13}], // Col 0
        [{suit: 3, rank: 10}, {suit: 0, rank: 6}, {suit: 0, rank: 1},
{suit: 0, rank: 9}, {suit: 0, rank: 5}, {suit: 0, rank: 12}, {suit: 0, rank:
4}], // Col 1
        [{suit: 2, rank: 11}, {suit: 3, rank: 4}, {suit: 3, rank: 11},
{suit: 1, rank: 5}, {suit: 2, rank: 6}, {suit: 3, rank: 7}, {suit: 1, rank:
3}], // Col 2
        [{suit: 2, rank: 7}, {suit: 2, rank: 4}, {suit: 2, rank: 2},
{suit: 1, rank: 2}, {suit: 0, rank: 2}, {suit: 3, rank: 11}, {suit: 0, rank:
13}], // Col 3
        [{suit: 3, rank: 12}, {suit: 3, rank: 1}, {suit: 0, rank: 7},
{suit: 1, rank: 6}, {suit: 2, rank: 9}, {suit: 2, rank: 3}, {suit: 0, rank:
3}], // Col 4
        [{suit: 2, rank: 3}, {suit: 3, rank: 1}, {suit: 3, rank: 8},
{suit: 0, rank: 3}, {suit: 3, rank: 10}, {suit: 1, rank: 7}, {suit: 0, rank:
7}], // Col 5
        [{suit: 1, rank: 12}, {suit: 0, rank: 9}, {suit: 2, rank: 13},
{suit: 2, rank: 10}, {suit: 3, rank: 6}, {suit: 3, rank: 1}], // Col 6
        [{suit: 2, rank: 8}, {suit: 0, rank: 10}, {suit: 3, rank: 5},
{suit: 3, rank: 11}, {suit: 0, rank: 10}, {suit: 3, rank: 4}, {suit: 1, rank:
4}] // Col 7
    ],
    cells: [null, null, null, null],
    foundations: [null, null, null, null]
};

let draggedCardElement = null;
let draggedCardData = null;

// --- DOM Elements ---
const findDeadlocksBtn = document.getElementById('findDeadlocks');
const showHintBtn = document.getElementById('showHint');
const resetBoardBtn = document.getElementById('resetBoard');
const hintBox = document.getElementById('hintBox');

// --- Functions ---

/**
 * Converts card data to a string rank
 */
function getRankStr(rank) {
    if (rank === 1) return 'A';
    if (rank === 11) return 'J';
    if (rank === 12) return 'Q';
    if (rank === 13) return 'K';
    return rank.toString();
}

```

```

    }

    /**
     * Converts card data to a suit character and color
     */
    function getSuitInfo(suit) {
        switch (suit) {
            case 0: return { char: '♥', color: 'is-red' };
            case 1: return { char: '♦', color: 'is-red' };
            case 2: return { char: '♣', color: 'is-black' };
            case 3: return { char: '♠', color: 'is-black' };
            default: return { char: '?', color: 'is-black' };
        }
    }
}

/**
 * Creates a card DOM element
 */
function createCard(cardData) {
    const { suit, rank } = cardData;
    const rankStr = getRankStr(rank);
    const { char: suitChar, color } = getSuitInfo(suit);

    const card = document.createElement('div');
    card.className = `card ${color}`;
    card.draggable = true;
    // Store data on the element for logic
    card.dataset.suit = suit;
    card.dataset.rank = rank;

    card.innerHTML = `
        <div class="card-top">${rankStr}${suitChar}</div>
        <div class="card-bottom">${rankStr}${suitChar}</div>
    `;

    // Add drag listeners
    card.addEventListener('dragstart', handleDragStart);
    card.addEventListener('dragend', handleDragEnd);

    return card;
}

/**
 * Renders the entire board from the initial state
 */
function renderBoard() {
    // Clear all columns, cells, and foundations

```

```

        document.querySelectorAll('.column, .cell,
.foundation').forEach(col => {
            col.innerHTML = '';
        });
        // Add back foundation suit icons
        document.querySelectorAll('.foundation').forEach((f, i) => {
            const { char: suitChar } = getSuitInfo(i);
            f.innerHTML = `<span class="suit-icon" style="font-size: 4rem;
color: rgba(255, 255, 255, 0.1);">${suitChar}</span>`;
        });

        // Render columns
        initialBoardState.columns.forEach((colData, colIndex) => {
            const colEl = document.getElementById(`column-${colIndex}`);
            colData.forEach(cardData => {
                colEl.appendChild(createCard(cardData));
            });
        });

        clearHighlights();
        hintBox.style.display = 'none';
    }

    /**
     * Clears all highlights from cards
     */
    function clearHighlights() {
        document.querySelectorAll('.card').forEach(card => {
            card.classList.remove('highlight-problem', 'highlight-
blocker');
        });
    }

    /**
     * Finds and highlights deadlocks based on DOM
     */
    function findDeadlocks() {
        clearHighlights();
        let cycleCount = 0;

        document.querySelectorAll('.column').forEach((colEl, colIndex) =>
{
            const cards = Array.from(colEl.querySelectorAll('.card'));

            for (let j = 0; j < cards.length; j++) {
                const cardX = cards[j];
                const cardXSuit = parseInt(cardX.dataset.suit);
                const cardXRank = parseInt(cardX.dataset.rank);

```

```

        for (let k = j + 1; k < cards.length; k++) {
            const cardY = cards[k];
            const cardYSuit = parseInt(cardY.dataset.suit);
            const cardYRank = parseInt(cardY.dataset.rank);

            // The deadlock condition from the C++ code
            if (cardXSuit === cardYSuit && cardXRank < cardYRank)
        {
                cycleCount++;
                console.log(`Cycle ${cycleCount} found in Col
${colIndex}: ${getRankStr(cardXRank)}${getSuitInfo(cardXSuit).char} is blocked
by ${getRankStr(cardYRank)}${getSuitInfo(cardYSuit).char}`);
                cardX.classList.add('highlight-problem');
                cardY.classList.add('highlight-blocker');
            }
        }
    });

    if (cycleCount === 0) {
        alert("No 1-suit deadlocks found on this board!");
    } else {
        alert(`Found ${cycleCount} 1-suit deadlock(s)! Check the
highlighted cards.`);
    }
}

// --- Drag and Drop Handlers ---

function handleDragStart(e) {
    // Only allow dragging the last card in a column or a card in a
cell
    const isLastInColumn =
e.target.parentElement.classList.contains('column') && e.target ===
e.target.parentElement.lastElementChild;
    const isInCell =
e.target.parentElement.classList.contains('cell');

    if (isLastInColumn || isInCell) {
        draggedCardElement = e.target;
        draggedCardData = e.target.dataset;
        setTimeout(() => e.target.classList.add('dragging'), 0);
    } else {
        e.preventDefault(); // Don't allow drag
    }
}

```

```

function handleDragEnd(e) {
  e.target.classList.remove('dragging');
  draggedCardElement = null;
  draggedCardData = null;
  document.querySelectorAll('.drop-target').forEach(e1 =>
e1.classList.remove('drop-target'));
}

function handleDragOver(e) {
  e.preventDefault(); // Necessary to allow drop
  const dropzone = e.target.closest('.dropzone');
  if (dropzone && isValidDrop(dropzone)) {
    dropzone.classList.add('drop-target');
  }
}

function handleDragLeave(e) {
  const dropzone = e.target.closest('.dropzone');
  if (dropzone) {
    dropzone.classList.remove('drop-target');
  }
}

function handleDrop(e) {
  e.preventDefault();
  const dropzone = e.target.closest('.dropzone');
  if (dropzone && isValidDrop(dropzone)) {
    // If it's a foundation or cell, it must be the direct target
    if (dropzone.classList.contains('cell') ||
dropzone.classList.contains('foundation')) {
      // Remove placeholder icon
      if (dropzone.classList.contains('foundation')) {
        dropzone.innerHTML = '';
      }
      dropzone.appendChild(draggedCardElement);
    }
    // If it's a column, append to the end
    else if (dropzone.classList.contains('column')) {
      dropzone.appendChild(draggedCardElement);
    }
    // If it's a card in a column, append after it
    else if (e.target.closest('.card') &&
e.target.closest('.column')) {
      e.target.closest('.column').appendChild(draggedCardElemen
t);
    }
  }
  if (dropzone) {

```

```

        dropzone.classList.remove('drop-target');
    }
}

function isValidDrop(dropzone) {
    if (!draggedCardElement) return false;

    const targetType = dropzone.dataset.type;
    const cardRank = parseInt(draggedCardData.rank);
    const cardSuit = parseInt(draggedCardData.suit);
    const cardIsRed = cardSuit === 0 || cardSuit === 1;

    // Case 1: Dropping on a Free Cell
    if (targetType === 'cell') {
        // A cell can only have one card
        const children = Array.from(dropzone.children).filter(c =>
c.classList.contains('card'));
        return children.length === 0;
    }

    // Case 2: Dropping on a Foundation
    if (targetType === 'foundation') {
        const foundationSuit = parseInt(dropzone.dataset.suit);
        if (cardSuit !== foundationSuit) return false; // Must match
suit

        const topCard = dropzone.lastElementChild;
        if (!topCard || topCard.classList.contains('suit-icon')) {
            return cardRank === 1; // Only Aces on empty foundations
        } else {
            const topCardRank = parseInt(topCard.dataset.rank);
            return cardRank === topCardRank + 1; // Must be sequential
        }
    }

    // Case 3: Dropping on a Column (either empty or on another card)
    if (targetType === 'column') {
        const topCard = dropzone.lastElementChild;
        if (!topCard) {
            return true; // Any card can move to an empty column
        } else {
            // Check logic for dropping on the card
            const topCardRank = parseInt(topCard.dataset.rank);
            const topCardSuit = parseInt(topCard.dataset.suit);
            const topCardIsRed = topCardSuit === 0 || topCardSuit ===
1;

            // Rank must be one less

```

```

        if (cardRank !== topCardRank - 1) return false;
        // Color must alternate
        if (cardIsRed === topCardIsRed) return false;

        return true;
    }
}
return false;
}

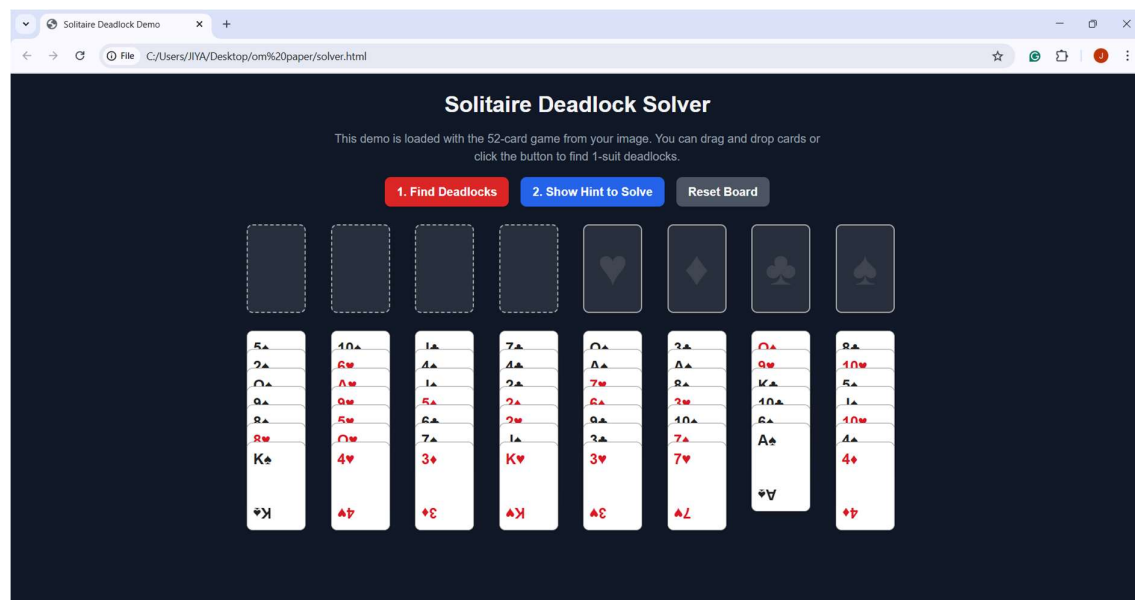
// --- Event Listeners ---
findDeadlocksBtn.addEventListener('click', findDeadlocks);
showHintBtn.addEventListener('click', () => {
    hintBox.style.display = hintBox.style.display === 'block' ? 'none'
: 'block';
});
resetBoardBtn.addEventListener('click', renderBoard);

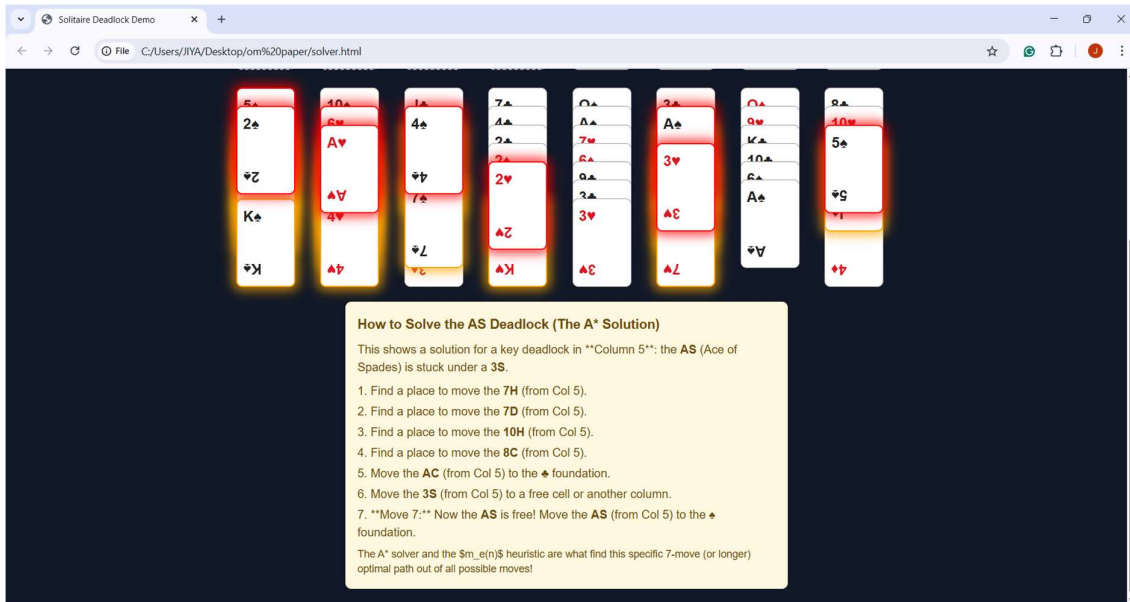
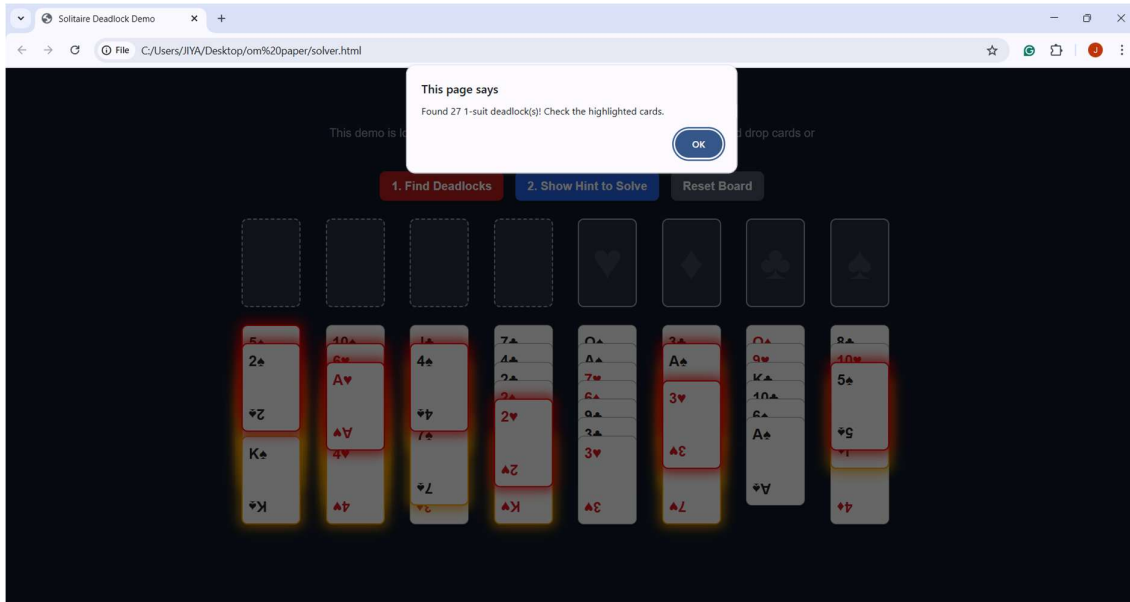
// Add drop listeners to all dropzones
document.querySelectorAll('.dropzone').forEach(zone => {
    zone.addEventListener('dragover', handleDragOver);
    zone.addEventListener('dragleave', handleDragLeave);
    zone.addEventListener('drop', handleDrop);
});

// Initial render
renderBoard();
</script>
</body>
</html>

```

OUTPUT:





6. Conclusion

The paper successfully demonstrates the first-known effective method for finding provably optimal solutions to skill-based solitaire games like FreeCell, which are known to be NP-hard. The core innovation is a strong, admissible heuristic for the A* search algorithm. This heuristic is derived by mapping the game state to a directed graph, where "deadlock" situations (which make optimal solutions hard to find) are represented by cycles.

The authors' method calculates the *minimum number of moves* required to break these cycles, adding this value to the heuristic's estimate. This provides a highly accurate estimate of the remaining moves, guiding the A* search efficiently. The best-performing version of their solver, h_2 , balances accuracy and speed by considering only 1-suit and 2-suit cycles.

7. References

G. Paul and M. Helmert, "Optimal Solitaire Game Solutions using A* Search and Deadlock Analysis" in *Proc. 9th Annu. Symp. Combinatorial Search (SoCS)*, Tarrytown, NY, USA, 2016, pp. 135-136. (<https://ai.dmi.unibas.ch/papers/paul-helmert-icaps2016wshsdip.pdf>)